Indian Institute of Information Technology Allahabad

Department of Information Technology

Cyber Security (IT-30001) | Autumn Semester 2026

Assignment: Web Server Security — Exploitation and Mitigation of Four Vulnerability Categories

Instructor: Dr. Soumyadev Maity | **Group:** Group 10

TABLE 3. OWNERSHIP DECLARATION

Member Name	Roll Number	Category	Variant Implemented	Directory
Samir Kumar	IIT2024274	Cross-Site Request Forgery (CSRF)	Variant 2: POST-based forged action (auto-submitting form)	`IIT2024274(CSRF)/`
Sathvik Boina	IIT2024263	OS Command Injection	Variant 1: In-band (command output returned)	`IIT2024263(Command_Injection)/`
G.P. Chandra Sekhar	IIT2024257	SQL Injection (SQLi)	Variant 1: Authentication bypass	`IIT2024257(SQLi)/`
Ashish Bairwa	IIT2024261	Cross-Site Scripting (XSS)	Variant 2: Reflected XSS	`IIT2024261(Reflected_XSS)/`

DETAILED VULNERABILITY REPORTS (TABLE 2 REQUIREMENTS)

1. Cross-Site Request Forgery (CSRF)

Implemented by: **Samir Kumar (IIT2024274)** | Variant 2: POST-based auto-submitting form

(a) Vulnerable Implementation

The banking transfer endpoint in `server.js` accepts state-changing transactions based exclusively on session cookie authentication without checking proof of user intent:

```
bankApp.post('/transfer', requireAuth, (req, res) => {
  const sender = req.session.username;
  const { recipient, amount } = req.body;
  // Flaw: Executes transfer blindly without verifying CSRF tokens
  const result = db.transferFunds(sender, recipient, amount);
  return res.redirect('/dashboard');
});
```

(b) Working Exploit

The attacker hosts an external page on `http://localhost:4000/claim-prize` containing an invisible auto-submitting POST form:

```
<form id="csrfPayloadForm" action="http://localhost:3000/transfer" method="POST">
  <input type="hidden" name="recipient" value="attacker_bob" />
  <input type="hidden" name="amount" value="1000" />
</form>
<script>document.getElementById('csrfPayloadForm').submit();</script>
```

Execution Flow: Alice is logged in at `localhost:3000`. When she visits `localhost:4000`, her browser automatically sends her session cookie with the cross-origin POST request. The bank transfers $1,000 to Bob Malory without Alice's knowledge or consent.

(c) Root Cause

The application accepted a state-changing POST request relying entirely on ambient browser credentials (session cookies transmitted automatically on cross-site requests) without verifying unpredictable, secret proof of user intent generated by the legitimate origin.

(d) Implemented Mitigation

Anti-CSRF Synchronizer Token Pattern: The server generates a cryptographically random, unpredictable token per session (`crypto.randomBytes(24).toString('hex')`) embedded as a hidden form field. On POST, the server validates:

```
if (currentMode === 'MITIGATED') {
  const sessionToken = req.session.csrfToken;
  const submittedToken = req.body._csrf;
  if (!submittedToken || submittedToken !== sessionToken) {
    return res.status(403).send("403 Forbidden: Invalid or missing CSRF token");
  }
}
```

Because the attacker domain cannot read Alice's token under the Same-Origin Policy (SOP), the forged request arrives without a valid token and is rejected with HTTP 403 Forbidden.

(e) Other Mitigations (Theory)

- **SameSite Cookies:** Configuring cookies with `SameSite=Strict` or `SameSite=Lax` prevents browsers from attaching session cookies to cross-site POST requests.
- **Re-Authentication & Step-Up Verification:** Requiring password re-entry, 2FA OTP, or CAPTCHA before executing sensitive transactions.

- **Custom Headers & Preflighted CORS:** Requiring custom HTTP headers (e.g., `X-Requested-With`) on API requests, triggering CORS preflight checks that HTML forms cannot bypass.
- **Avoiding State-Changing GETs:** Enforcing safe HTTP semantics (RFC 7231) by ensuring GET requests are strictly read-only.

2. OS Command Injection

Implemented by: **Sathvik Boina (IIT2024263)** | Variant 1: In-band (command output returned)

(a) Vulnerable Implementation

A diagnostic route invokes a system shell command by concatenating unvalidated user input directly into the command string:

```
const { exec } = require('child_process');
app.post('/ping', (req, res) => {
  const host = req.body.host;
  // Flaw: Passing raw user input to shell interpreter
  exec(`ping -c 2 ${host}`, (err, stdout, stderr) => {
    res.send(`<pre>${stdout || stderr}</pre>`);
  });
});
```

(b) Working Exploit

Payload: `127.0.0.1; cat /etc/passwd`

Execution: The shell interpreter parses the semicolon (`;`) as a command separator, executes the ping command, and subsequently executes `cat /etc/passwd`, returning the system password file content directly in the HTTP response.

(c) Root Cause

The application passes untrusted input directly to an operating system shell interpreter (`sh`, `bash`), allowing command chaining metacharacters (`;`, `&`, `|`, `` ` ``, `$()`) to alter command execution flow.

(d) Implemented Mitigation

Avoid the Shell via Parameterized Execution (`execFile` with Argument Vector):

```
const { execFile } = require('child_process');
app.post('/ping', (req, res) => {
  const host = req.body.host;
  // Direct binary execution: host treated strictly as an argument literal
  execFile('/sbin/ping', ['-c', '2', host], (err, stdout, stderr) => {
    res.send(`<pre>${stdout || stderr}</pre>`);
  });
});
```

When injecting metacharacters, the binary receives the entire string as a single literal argument without spawning a shell, returning: *ping: cannot resolve host: Unknown host*.

(e) Other Mitigations (Theory)

- **Allow-list Input Validation:** Enforcing strict regex validation (e.g., IPv4/IPv6 address patterns), rejecting any shell metacharacters.
- **Least-Privilege Execution:** Running the web server under an unprivileged user account (e.g., `www-data` with `nologin`) to limit execution scope.
- **Native Language APIs:** Utilizing built-in language networking functions (e.g., `dns.lookup`, `net.isIP`) rather than spawning OS binaries.

3. SQL Injection (SQLi)

Implemented by: **G.P. Chandra Sekhar (IIT2024257)** | Variant 1: Authentication bypass

(a) Vulnerable Implementation

The authentication handler dynamically builds SQL queries by concatenating raw user input:

```
const query = `SELECT * FROM users WHERE username = '${username}' AND password =
'${password}'`;
db.get(query, (err, row) => {
  if (row) {
    req.session.user = row.username;
    return res.redirect('/dashboard');
  }
  return res.status(401).send("Invalid credentials");
});
```

(b) Working Exploit

Payload: `admin' --`

Executed Query: `SELECT * FROM users WHERE username = 'admin' --' AND password = '...'`

The comment operator (`--`) causes the database engine to ignore the remainder of the query (the password check), allowing an unauthenticated attacker to log in as the administrator.

(c) Root Cause

The application constructs database queries by directly interpolating unvalidated, unsanitized user-supplied strings into the SQL command text, allowing attacker input to alter the syntax and structure of the query.

(d) Implemented Mitigation

Parameterized Queries (Prepared Statements):

```
const query = `SELECT * FROM users WHERE username = ? AND password = ?`;
db.get(query, [username, password], (err, row) => {
  // Input treated strictly as literal data parameters, never executable SQL
});
```

When injecting `admin' --`, the database treats the string as a literal username value. No user exists with that exact name, and login fails with 401 Unauthorized.

(e) Other Mitigations (Theory)

- **Input Validation with Allow-lists:** Validating input format against strict alphanumeric patterns, rejecting characters like quotes and dashes.
- **Least-Privilege Database Accounts:** Configuring the application database user with only essential read/write permissions, disabling DDL operations.
- **Safe Error Handling:** Suppressing raw database errors and stack traces from being returned to clients to prevent reconnaissance.

4. Cross-Site Scripting (Reflected XSS)

Implemented by: **Ashish Bairwa (IIT2024261)** | Variant 2: Reflected XSS

(a) Vulnerable Implementation

The server receives search parameters and reflects them directly into the response HTML without output encoding:

```
app.get('/search', (req, res) => {
  const query = req.query.q || '';
  res.send(`<h1>Search Results</h1><p>You searched for: ${query}</p>`);
});
```

(b) Working Exploit

Payload URL: `http://localhost:3000/search?q=<script>alert(document.cookie)</script>`

When a victim accesses the link, the browser executes the injected script within the application's origin context, enabling session cookie theft or page manipulation.

(c) Root Cause

The application reflects untrusted user input directly in its rendered HTML output without performing context-aware output encoding, causing the client browser to execute data as code.

(d) Implemented Mitigation

Context-Aware HTML Entity Encoding:

```
function escapeHTML(str) {
  return str.replace(/[&<>'"]/g, tag => ({
    '&': '&amp;', '<': '&lt;', '>': '&gt;', "'": '&#39;', '"': '&quot;'
  }[tag] || tag));
}
app.get('/search', (req, res) => {
  const query = escapeHTML(req.query.q || '');
  res.send(`<h1>Search Results</h1><p>You searched for: ${query}</p>`);
});
```

The characters `<` and `>` are converted to `<` and `>`, rendering safely as literal text.

(e) Other Mitigations (Theory)

- **Framework Auto-Escaping:** Leveraging modern frontend engines (e.g., React, Angular, EJS) that auto-escape dynamic variables.
- **HTML Sanitization:** Using DOMPurify to strip dangerous elements when rich HTML input is required.
- **Content Security Policy (CSP):** Setting `Content-Security-Policy: default-src 'self'` to disable inline scripts.
- **HttpOnly Cookie Flag:** Restricting client-side JavaScript from accessing session cookies via `document.cookie`.
- **Safe DOM Sinks:** Using safe DOM APIs (`textContent`) instead of dangerous sinks (`innerHTML`, `eval`).